

Homework 3 — Dynamic Programming

Due date: Friday, October 9th at 11:59PM (Central Time)

Instructions

This assignment must be submitted on Gradescope. You will upload your written answers as a PDF. Make sure your work is clearly labeled. When you upload, you must match each problem with the page(s) it is on.

Homework solutions are required to be typeset. Be sure to read the Homework Policies page of the syllabus, especially to see the suggested format for describing algorithms. Our solutions for any individual problem will fit in approximately one page or less. **If you submit an answer substantially longer than 2 pages, the TAs are allowed to stop reading at the end of page 2.**

You must clearly cite all sources and collaborators at the end of your submitted PDF (citing a source *does not* give you permission to copy verbatim from that cited source). Please see the [Course Syllabus](#) for the [full collaboration](#) and [late policies](#).

Undergraduates: For this and all future homework assignments, groups of up to 3 undergraduate students can submit joint solutions. All students in the group must collaborate on each problem.

Graduates: For this and all future homework assignments, graduate students must submit individual solutions.

1. Assymmetric Edit Distance

[3 points]

For this problem, we're going to redefine edit distance as the smallest sum of penalties for the operations below to convert one string into another. Note that unlike the version we saw in class, our definition won't be symmetric since deleting a character will be less of a penalty than inserting one:

- 2 points for **deleting a character** from the first string x .
- 3 points for **deleting 2 consecutive characters** from the first string x .
- 3 points for **inserting a character** from the second string y into the first string x .
- 4 points for **substituting a character** from the first string x with a character from the second string y .

(In class we had the same penalty of 1 for unaligned characters in either string). Matching a character in x with the same character in y is still free (0 points)

(a) Update the recurrence from class to calculate the new edit distance.

(b) Build the table to evaluate the recurrence **and state the new edit distance** from the string "spared" to the string "pair" (make sure to go in the correct direction – because it's assymmetric now, you'll get a different answer in each direction).

2. Cookie Connoisseur

[7 points]

Background. Congrats! You've been hired as a CS 401 TA. As a new TA, Omar has asked you to pick-up cookies for the staff meeting. However, the bakery only sells cookies in specific batch sizes, one batch per box. Since you have limited room, you want to find the least number of boxes to pick-up.

Specifically, you are given:

- a list of the integer batch sizes S – sorted so that $S[i] < S[i + 1]$ for all i , and $S[1] = 1$ (i.e. you can always buy a single cookie, so any order can be satisfied).
- an integer n , the total number of cookies Omar wants

For example, a set of batch sizes could be $S = [1, 4, 7, 8, 13]$, and Omar might ask you to pick up $n = 11$ cookies (which you can acquire by purchasing 2 batches).

You can only purchase whole batches of cookies, i.e., you can buy a batch of 8 cookies or a batch of 13 cookies, but you cannot buy a batch of 10 cookies.

Problem. You are going to design a dynamic programming algorithm that, given a list $S[]$ (of length b) which contains the number of cookies you can buy per batch, and an integer n number of cookies to buy, produces the **minimal number of batches** you need to buy to purchase **exactly** n cookies.

(a) The first thing that comes to mind is an algorithm like this:

```
1: function COUNTBOXES( $S[], n$ )
2:   numBoxes  $\leftarrow 0$ 
3:   for  $i$  from length( $S$ ) down to 1 do
4:     while  $n \geq S[i]$  do
5:       numBoxes  $\leftarrow$  numBoxes + 1
6:        $n \leftarrow n - S[i]$ 
7:   return numBoxes
```

This algorithm *does not* use a dynamic programming approach; instead, it tries to be greedy, but it **doesn't always work**. Come up with a list of batch sizes and total cookies where this algorithm fails, i.e. it uses more boxes than necessary. Describe the batch sizes found by the algorithm and a way to get a smaller total number of batches of cookies.

- (b) Write a recurrence (or multiple recurrences) that you can use to calculate the minimum number of boxes required to store n total cookies. (*Do not write code or pseudocode, just the recurrence function. You only need the total number of batches purchased, not the number per batch size.*)
- (c) Give a clear English description of what your recurrence(s) is calculating (*especially what the parameter(s) in your recurrence represent*) and state the settings of the parameter(s) to calculate for n cookies.
- (d) Evaluate your recurrence on the example you wrote in part (a). Make sure you get the correct answer! It's easy to accidentally write a recurrence that is actually just doing the code from part (a). (You don't need to submit anything for this part.)
- (e) Describe a memoization structure for your recurrence(s).
- (f) Give a filling order for your memoization structure. (*You do not have to give the pseudocode to fill it, just an order you could fill it in*).
- (g) Give the big- Θ running time of your algorithm, in terms of a number of cookies n and b = the length of S .

3. School Satisfaction [7 points]

Background. Miya often dislikes going to in-person lectures because in the past she's felt a lack of enthusiasm from her professors. So, she's decided to enroll in a hybrid course that lists the lecture materials for the next n weeks and let's her choose for each week whether to go in-person or not.

Miya can predict for each week how satisfying the lectures will be, with a numerical score. But she's very picky, so sometimes it's a negative number. Her goal is to avoid going to class as much as possible, but there are some penalties for skipping weeks:

- If she skips classes for one week there's no penalty.
- Skipping classes for exactly two or three consecutive weeks makes her feel isolated, which causes a penalty of -20 points to her total satisfaction.
- If she skips classes for four or more consecutive weeks, she has to watch a LOT of recordings, which causes a penalty of $-\infty$. This means that she cannot skip 4 or more consecutive weeks, though it is possible for her to skip more than 4 weeks in total if she spreads them out enough.

Your goal is to calculate the maximum sum of her satisfaction scores.

Example. Suppose `satisfaction` = `[10, -3, -21, -8, 5]`. Then:

- Going to lecture weeks 1 and 5 in-person results in a score of **-5** ($10 + 5$ satisfaction, but with a -20 penalty for skipping three weeks in the middle).
- Going to lecture weeks 1, 2, and 5 in-person would result in a score of **-8**.
- Going to lecture weeks 1, 2, 4, and 5 would result in a score of **4**.

Your algorithm should return **4** (which is the maximum possible).

Suppose `satisfaction` = `[-100, -100, 40, -100, -100]`. Then going to class only during week 3 would result in a score of 0 ($+40$ from the array, but with two -20 penalties), whereas going to classes in weeks 2 and 4 would result in a score of -200 . (no penalties, but -200 from the array). Your algorithm should return **0**.

Problem. Formally, given an array of integers `satisfaction` such that `satisfaction[i]` gives the satisfaction of going to classes for week i , your goal is to design a dynamic program to return the **maximum sum of satisfaction values** subject to the week-skipping penalties above.

- (a) Give a clear English description of what your recurrence(s) is calculating (especially what the parameter(s) in your recurrence represent).
- (b) Write a recurrence (or multiple recurrences) that you will use to solve this problem.
Hint: You will likely want more than one recurrence.
- (c) Give a brief justification that your recurrence calculates the correct value; we're not looking for a formal proof, but a brief explanation of what each of the cases represent, and why that set of cases is exhaustive.
- (d) Describe a memoization structure (or structures) for your recurrence(s).
- (e) Describe a filling-order for your memoization structure. *If you wrote (iterative) pseudocode to fill the structure, what would the running time be? (Do not write actual pseudocode for this portion, we just want to know what the running time would be). Justify the runtime with 1-2 sentences.*
- (f) What is your final answer going to be? (where in the memoization structure do we look?)

4. Programming: Unit Expressions: [7 points]

A “unit expression” is an expression involving only:

- parentheses
- + (that is, addition)
- * (that is, multiplication)
- The number 1

$(1 + 1) * (1 + 1 + 1)$ is a unit expression that evaluates to 6. This expression has **5** copies of the number 1.

$1 + 1 + 1 + 1 + 1 + 1$ also is a unit expression that evaluates to 6. This expression has **6** copies of the number 1.

Your task is to write a program that given an integer n , returns the fewest number of copies of 1 that can appear in a unit expression evaluating to n . You do not have to return the expression itself. For example, the correct output for $n = 6$ would be **5** (corresponding to the number of 1's in the first expression above). If it is not possible to create a unit expression evaluating to n (such as for $n = -5$), your program should return **-1**.

For full credit, your algorithm must run in time at most $O(n\sqrt{n})$ for a given input n . There are stress-tests provided on gradescope. To meet the efficiency requirements you must:

- Pass all the stress tests
- Have a program which is theoretically at most $O(n\sqrt{n})$ (we will check for this by hand and adjust scores if you passed the stress-tests but did not meet the theoretical requirement). You will get partial credit as long as your algorithm runs in time at most $O(n^2)$.

A few details:

- Submit your code to the “Homework 3 Programming” assignment on gradescope. *Do not submit it with your writeup.*
- The starter file `UnitExpression.py` with a function stub is available on the course Blackboard. Don't change the name of the file (gradescope will look to compile an uploaded file of the same name) and don't change the function name or parameters.
- Even though this is a programming problem, we *highly recommend* that you walk through the dynamic programming process and design a recurrence for this problem, before trying to implement an iterative algorithm to solve it.
- Writing additional functions is allowed!

Extra Credit: Text Formatting [1 point]

You are not required to submit attempts at extra credit problems (they do not count toward the dropped/counted problems at the end of the quarter). They are not worth very much, and attempts will be graded harshly (i.e., you will not receive credit unless your solution is roughly correct). These are intended more as an extra challenge for anyone who wants one. You should not expect extra credit problems to have a significant impact on your grade. We do not recommend attempting them until you've done all the other problems on the assignment.

Background. In typesetting systems, like the LaTeX system in which this assignment is formatted, the input for paragraphs is given in blocks of plain text consisting of words and punctuation separated by blanks and new lines. Formatted paragraphs are produced by laying out this text using a fixed maximum width L per line by figuring out where to break each line of text and continue the paragraph on the next line. An innovation of the TeX system on which LaTeX is based was to phrase paragraph formatting as an optimization problem, which produced superior paragraph formatting and has been adopted by many other word processing systems.

We consider a simplified version of this optimization problem in which words are considered indivisible units of fixed width (no hyphenation to break single words up across multiple lines) and punctuation is considered part of the word is next to.

In our version, the input consists of a sequence of n positive real numbers $w_1, \dots, w_n$ representing the lengths that the words $1 \dots n$ would occupy in a given font in order, together with a positive real number b representing the *minimum* width of a blank space in that font, and another real number representing the allowed width L of each line. (You can assume that each $w_i \leq L$, i.e. no single word is too long to fit on a line.)

A legal formatting for this text consists of a sequence of integers $0 = e_0 < e_1 < \dots < e_k = n$ for some k , where the i -th line of the paragraph consists of the words numbered $e_{i-1} + 1, \dots, e_i$ separated by blank spaces. (That is, e_i is the number of the word that ends the i -th line in the paragraph and the number of blank spaces in a line is 1 less than the number of words in it.)

It is required that no line be longer than L , so if a line consists of words t through u , then

$$w_t + b + w_{t+1} + b + \dots + b + w_u \leq L$$

The *slack* of this line is the difference between the left and right sides of this inequality; in other words, the amount of extra space left on that line.

An optimum formatting for this input is one that *minimizes* the **sum** of the **squares** of the **slacks** of its lines.

Problem. Describe an efficient dynamic programming algorithm (using pseudocode) that finds an *optimal formatting* from the inputs $w_1, \dots, w_n$, b , and L .